

HONEYWELL SOFTWARE ENGINEERING TEAMS

AI Champions Programme

Module 06

Test Strategy, Testcontainers and End-to-End Validation

Deriving a test strategy directly from the SDD feature's acceptance criteria, validating against real service and database dependencies with Testcontainers, and proving the strategy catches a real defect.

	DURATION 2 Hours		FORMAT Hands-On Lab		DELIVERED FOR Honeywell Eng. Teams
---	----------------------------	--	-------------------------------	---	--

How We'll Spend These Two Hours

Six connected moves — from a spec-derived strategy to proof the strategy actually works.



01

Test Strategy, Derived From the Spec

15 MIN

Turning Module 04/05's acceptance criteria into a concrete, traceable test strategy.



02

The Test Pyramid for SDD Features

20 MIN

Layering unit, integration, API/contract, and E2E tests so the right things run at the right cost.



03

Testcontainers: Why Real Dependencies Matter

25 MIN

Moving from mocks and stubs to real, ephemeral service and database dependencies.



04

Test Data & Negative Scenarios

15 MIN

Designing test data that covers the happy path, edge cases, and deliberate failure scenarios.



05

Coverage & Acceptance-Criteria Validation

15 MIN

Confirming every acceptance criterion has a test, and every test maps back to a criterion.



06

Hands-On Lab: Inject a Defect, Prove the Strategy Catches It

30 MIN

Derive the strategy, build the tests, then deliberately break the feature to prove the safety net works.

Module 06 in the 12-Module Journey

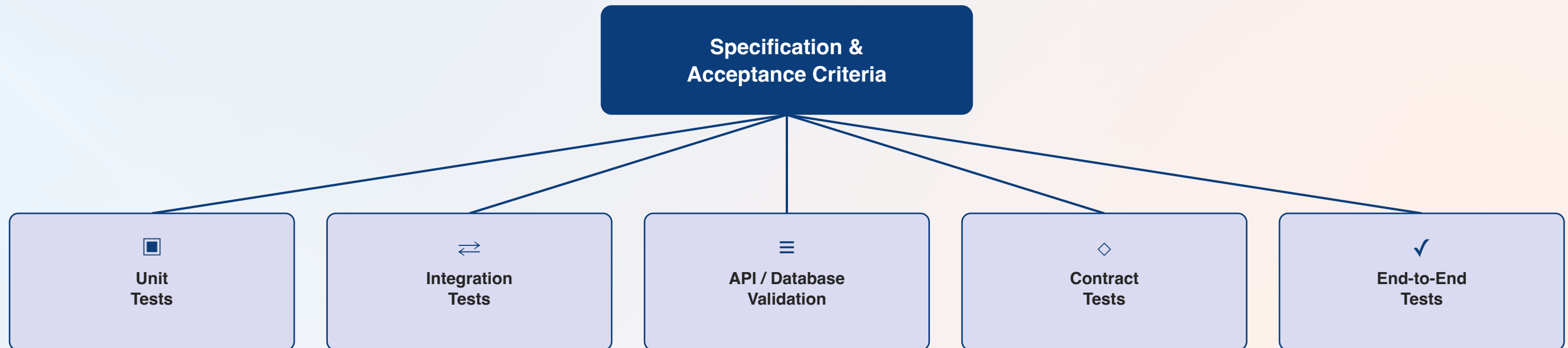
The feature Module 05 built now earns its release readiness — on evidence, not assumption.



Why this matters: Module 08's PR quality gates check for exactly this test evidence; Module 11's ROI model treats defects caught here as cost avoided, not cost incurred later.

From Acceptance Criteria to Test Strategy

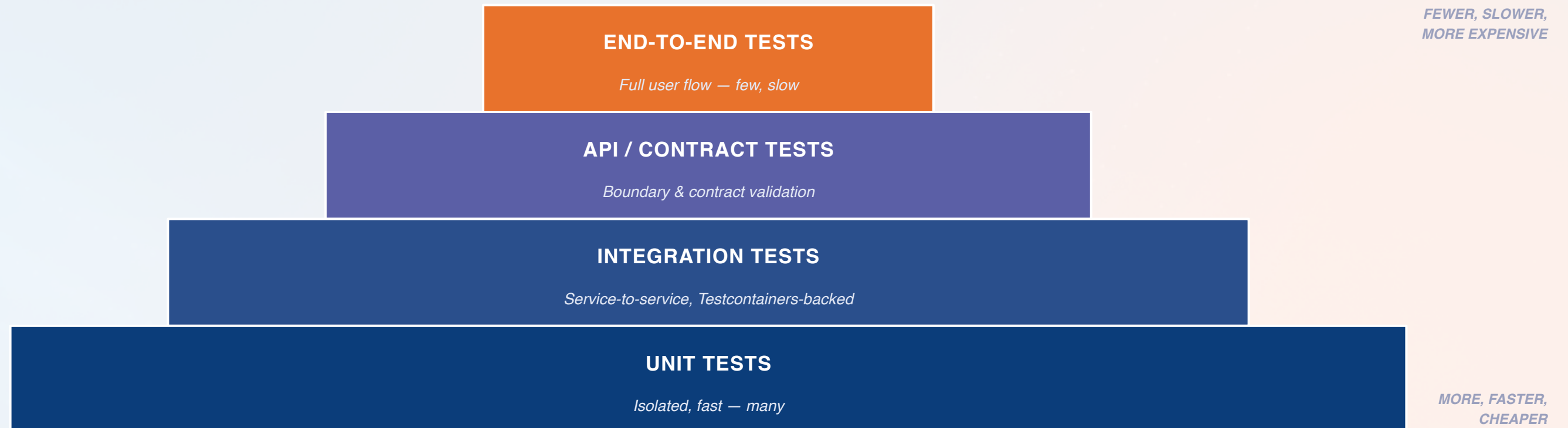
The specification doesn't just describe the feature — it defines what "tested" means for it.



Why this matters: A test strategy isn't a checklist chosen independently — every layer exists because some part of the specification needs a specific kind of proof.

The Test Pyramid for SDD Features

The right amount of each test type — not everything as an end-to-end test.



The throughline: Testcontainers is what makes the integration layer trustworthy — real dependencies, without paying full E2E cost for every scenario.

Testcontainers: Why Real Dependencies Matter

What changes when integration tests run against the real thing.

TESTCONTAINERS

● A real database/service, running in an ephemeral container

● The same behavior in CI as in production

● Versions pinned to match production dependencies

● Integration coverage that's actually trustworthy

MOCKS & STUBS

● Database behavior guessed at, not verified

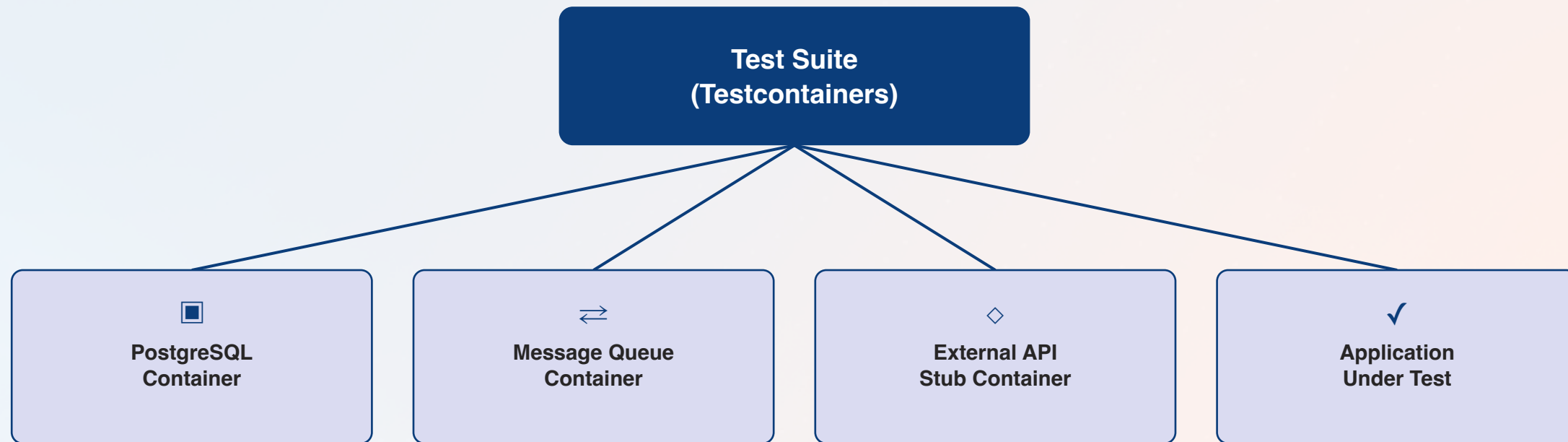
● Passes in CI, fails against the real service

● Config drift between mock and production accumulates silently

● False confidence in integration coverage

Testcontainers Architecture, at a Glance

What the test suite actually talks to while it runs.



Why this matters: Containers spin up fresh for each test run and tear down after — no shared state, no "works on my machine," no drift from production versions.

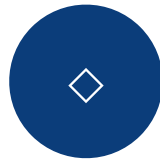
Test Data & Negative Scenarios

Four categories of input, because a feature is only as tested as its worst-case data.



Happy Path

Valid input, expected flow — confirms the feature does what the spec says it should.



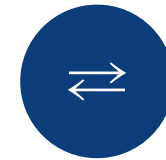
Boundary & Edge Cases

Empty inputs, maximum sizes, off-by-one conditions — where bugs like to hide.



Negative & Invalid Input

Malformed, unauthorized, or out-of-contract input — confirms the feature fails safely.



Failure & Timeout Scenarios

A dependency unavailable, slow, or erroring — confirms graceful degradation, not a crash.

Coverage & Acceptance-Criteria Validation

The same traceability discipline from Module 04, now checked against running tests.

Acceptance Criterion	Covered By	Status
AC-01: Valid order returns 201	test_create_order_valid (unit)	✓ Passing
AC-02: Duplicate order rejected	test_duplicate_order (integration)	✓ Passing
AC-03: Payment service timeout handled	test_payment_timeout (Testcontainers)	✓ Passing
AC-04: Full checkout flow completes	test_checkout_e2e (E2E)	✓ Passing

A specification with no unmapped acceptance criteria, and a test suite with no orphaned tests.

Hands-On Lab Preview: Prove the Strategy Catches It

The 30-minute activity that closes this module — a test suite tested against a real defect.



**Derive
Strategy**

Map the spec's acceptance criteria to test types across the pyramid.



**Write / Update
Tests**

Build or extend unit, integration, and E2E tests for the feature.



**Run With
Testcontainers**

Execute against real, ephemeral service and database dependencies.



**Inject a
Deliberate Defect**

Introduce a realistic bug into the implementation, on purpose.



**Confirm It's
Caught**

Re-run the suite and verify the defect is caught — not silently missed.

The throughline: A test strategy is only proven once it's caught something. This lab is that proof, not a formality.

Module 06 Outcomes & What's Next

- ✓ A test strategy has been derived directly from the specification's acceptance criteria
- ✓ The test pyramid — unit, integration, API/contract, E2E — is understood
- ✓ Testcontainers is understood as real dependencies over mocks, and why that matters
- ✓ Test data covering happy path, edge cases, and negative scenarios has been designed
- ✓ Every acceptance criterion is traceable to a specific, passing test
- ✓ A deliberate defect has been injected and caught, proving the strategy works



NEXT MODULE

Module 07 — MCP-Enabled Agentic Engineering Workflows and Reusable Engineering Skills (2 hours). Connecting agents to approved repositories, APIs, databases, and test infrastructure — including the Testcontainers-based tests just built — under clear governance.

Questions & Discussion

Module 06 — Test Strategy, Testcontainers and End-to-End Validation